EN.705.643 Final Project Proposal

Finding Climbable Rock Using Airborne Laser Scans

Zachary Freund Yixian Li

1 Introduction & Motivation

Rock climbers find new cliffs by word of mouth and by walking. Websites list the climbs that came out of that process, but nothing lists the cliffs nobody has looked at. What if a model could predict whether a location has climbable rock? Climbers could find new routes without blindly wandering around, and a similar model could identify cliff habitat for birds of prey for wildlife conservation.

LiDAR (light detection and ranging) scanners fire laser pulses from an aircraft and time their reflections, producing a point cloud: millions of 3D points mapping the shape of the terrain. Most of the United States has been scanned this way, and the scans are freely available through the U.S. Geological Survey (USGS) [1]. Here we ask whether a deep learning model can pick out climbable cliffs in those scans, and in particular whether it needs the raw 3D points or whether a simple top-down height map is enough. The point cloud is richer but more computationally expensive. A digital elevation model (DEM) flattens it into an image whose pixel values are ground height, which is smaller and easy to feed to an image network. We will train models on both and compare.

The biggest challenge is that we have to make our own labels. No dataset marks climbable rock in a scan, and the closest substitute, Mountain Project [2], where climbers record existing climbs, is only a rough guide. It gives one coordinate for a whole section of cliff, often 10–30 m from the rock, and it records only where people do climb, so it has no negative examples of cliffs that are unclimbable (e.g., due to rock type). Obvious negatives, such as the flat ground above a cliff and the rubble below it, look nothing like a cliff and make the task easier than it should be. Creating suitable labels is therefore an important part of the project.

2 Related Work

PointNet++ [3] is the deep neural network we will use for 3D input. It takes an unordered set of points, groups nearby points into small neighbourhoods, summarises each one, and repeats this at larger scales. It is usually pretrained on ModelNet40 [4], a collection of 3D models of everyday objects such as chairs and lamps. Li et al. [5] ran PointNet++ and two other 3D deep learning models on airborne scans to label each point as building, vegetation, or ground, beating a classical random-forest classifier.

In terms of using airborne data for detecting terrain features, Albrecht et al. [6] found ancient ruins in airborne scans from few, noisy labels, and Rafique et al. [7] found sinkholes, reporting that the gradient of the DEM works far better than aerial photos. Both flatten the scan to 2D first. That is reasonable for ruins and sinkholes, which are gentle shapes. We argue that cliffs are the case where flattening should hurt most and where the raw 3D point data might give us better results.

3 Data & models

We will gather data for several climbing areas in Arkansas. So far we have run a proof of concept on Horseshoe Canyon Ranch near Jasper; the other areas have not been chosen or downloaded yet.

3.1 Data:

USGS 3DEP scans: Free airborne LiDAR scans from the US Geological Survey's national programme [1]. In the proof of concept: the 2017 survey of the area, 18.66 M points over 1.5 × 1.7 km (7.3 per m²).

Mountain Project: A website where climbers record climbs [2], giving one coordinate per cliff section. In the proof of concept: 530 climbs at 33 coordinates.

Open Climbing:

Application for mapping climbing areas and creating interactive climbing guides[11]. This website is an open source place to record climbing areas and specific route locations. Open Climbing utilizes OpenStreetMap for the map engine backing.

Training examples: Locations we label as climbable or not, each with the scan around it. In the proof of concept: 354 labelled as climbable and 1,093 not, from 30 cliff sections.

3.2 Pretrained models:

For 3D, a PointNet++ [3] checkpoint trained on ModelNet40 [4] (from the open-source yanx27 implementation [8], MIT licence). For 2D, a ResNet-18 [9] trained on ImageNet [10].

3.3 Feasibility:

We ran the proof of concept to assess feasibility. The scan, the climb list and the PointNet++ checkpoint were all downloaded and loaded locally. The model also appeared to run as expected. The pretrain checkpoint loads cleanly, and a two-epoch trial trains and evaluates end to end.

4 Experiments

4.1 Creating the Dataset: Are we labeling correctly?

To address the challenge of scaling our dataset, we propose a semi-supervised labeling pipeline that leverages a small baseline of high-precision route examples to generate self-supervised pseudo-labels. Before applying this method at scale, we will rigorously validate the efficacy of our heuristic by computing the Euclidean distance error between the generated labels and the precise GPS coordinates of known routes. By plotting this error utilizing a cumulative distribution function (CDF), we will determine the proportion of pseudo-labels that fall within an acceptable threshold of the true starting points. To empirically evaluate the viability of this labeling approach, we will conduct a comparative analysis across three networks: a baseline model trained exclusively on the high-precision data, a second model trained solely on the heuristic-generated pseudo-labels, and a third model pre-trained on the pseudo-labels and fine-tuned on the precise dataset. Ultimately, if the model relying strictly on heuristic data demonstrates catastrophic performance degradation compared to the baseline, we will iteratively refine our labeling heuristic prior to full-scale dataset generation and training.

4.2 Determining the Proper Confidence Threshold

To determine the optimal confidence threshold for P(Climbable), we propose a dual-metric evaluation strategy designed to accurately estimate the number of viable climbing routes within a given region. First, employing a "bag of routes" methodology, we will run our trained model across broad climbing areas and cluster the resulting predictions using the approximate GPS coordinates of known crags as centroids. We will then compare the predicted route counts per cluster against ground-truth totals, calculating the Mean Absolute Percentage Error (MAPE) across dozens of crags to identify a threshold that reliably predicts route volume independent of exact spatial localization. To complement this, we will analyze the density gradient decay of predicted routes relative to the crag centroids. By precalculating the true route density gradients from our training areas, we can iteratively modulate the confidence threshold until our predicted distributions align with these baseline values, effectively filtering out hallucinated predictions. By jointly minimizing the cluster-based MAPE and optimizing for spatial density alignment, we will establish a robust, empirically validated confidence threshold for our model.

4.3 Geometric Baseline Comparison

It is easy for a model to just learn "vertical plane = climbing route." Compare our deep learning model against a simple geometric algorithm for example using RANSAC to find planes or thresholding LiDAR normal vectors <75 degrees. Our model must outperform this baseline to prove it is learning specific climbable features, not just cliff faces.for

4.4 Comparing 3-D Point Cloud Data to 2.5D Elevation Gradient Model

To rigorously justify the computational overhead of our 3D LiDAR point cloud model, we propose a direct comparative framework evaluating it against a lightweight 2.5D baseline. To establish this robust control, we will adapt a validated top-down 2D U-Net architecture utilizing 2-channel elevation gradients and weighted cross-entropy loss to account for the severe class imbalance of vertical terrain. We will then execute a one-to-one comparative "shootout" between the 2D and 3D architectures across three primary experiments: an ablation study testing each model's robustness to heuristic pseudo-label noise, a spatial clustering analysis optimizing confidence thresholds to align our predicted "bag of routes" with true Mountain Project route counts, and a final predictive baseline comparison against simple geometric heuristics. By analyzing their respective distance error distributions and clustering metrics, this experimental framework will definitively prove whether processing raw 3D spatial geometry is necessary to detect climbable rock, or if a highly efficient 2D approximation yields comparable results.

5 Training Plan

5.1 Data scope and splits

We will use three to four climbing areas. Each input is the column of points within 5 m of a location, resampled to 1,024 points and scaled into a unit sphere, the same way as ModelNet40. Splits are made by whole crag or wall, never by individual route: neighboring route cylinders are 3 m apart and overlap, so a random split would leak near-copies into the test set. The exact split will be settled as the data comes in, but the plan has two held-out test sets:

Precise test set: We will hold out about 25% of the precise OpenClimbing route locations, which are scattered across the USA. They give exact GPS ground truth from many rock types and regions, so they are the strictest test of whether the model generalises. The other 75% are used for training and validation.

Known-area test set: We will train on two main sections of Horseshoe Canyon Ranch (HCR) and hold out a third crag there. No precise route locations exist for HCR, so we will collect GPS positions for that crag's routes by hand. This shows how the model performs, including a full probability map, on a well-known area that we can check in person.

Neither test set is used for training or for choosing a model.

5.2 Loss, optimizer, schedule

Loss: Cross-entropy weighted by inverse class frequency. There are about 3 negatives for every positive, and the weighting stops the model from always predicting "not climbable".

Optimizer: AdamW, with learning rate 10^{-3} for the new head and 10^{-4} ($0.1\times$) for the pretrained backbone, and weight decay 10^{-4}. The backbone moves slowly because its pretrained filters are still useful, while the new head needs time to catch up. We compare three settings: scratch, frozen backbone, and full fine-tune.

Schedule: Cosine annealing over 60 epochs with batch size 32. Training stops early after 12 epochs without improvement, and we keep the checkpoint with the best validation balanced accuracy, using AUROC to break ties. With only about 1–5k patches the model overfits quickly, and early stopping limits this.

5.3 Augmentation

Each time a patch is read, we draw a new random set of 1,024 points from it. We also rotate it at random about the vertical axis only, scale it by 0.9–1.1, and add jitter ($\sigma = 0.01$, clipped at 0.03). We do not tilt the patches, because a cliff's direction relative to gravity is part of what makes it climbable.

5.4 Metrics

The main metrics are balanced accuracy and AUROC, which are not misled by the class imbalance. We also report precision, recall, F1 and the confusion matrix. We break test accuracy down by negative type (talus, plateau, random) and break positive recall down by how much of the route height the LiDAR covered, to see whether errors come from the model or from parts of the cliff the scanner never saw. Because the held-out HCR crag is small, every result is repeated over 3 split seeds and reported as mean ± std. The label-level metrics from Sec. 4 (distance-error CDF, cluster MAPE) are computed on top of these.

5.5 Compute and time estimate

Everything runs locally on one RTX 3080 (10 GB) with 16 CPU threads and 19 GB RAM, so no cloud is needed. We measured about 260 training patches/s (batch 32, 2.9 GB of GPU memory) and about 380 patches/s for inference. With 4 areas (about 5k patches) an epoch takes about 15 s, and a 60-epoch run takes 15–20 min (proof of concept on HCR alone: under 5 min). The full set of runs is 3 modes × 3 label settings (precise only, pseudo-labels only, pseudo-labels then precise) × 3 seeds = 27 runs, or about 8 h of GPU time, which fits in one overnight run. The 2D model on DEM tiles adds about 1 h. Preparing the data costs more than training: each area's scan takes a few minutes to download and process, plus about 30 min for all the OpenClimbing tiles. A 4 m probability map over HCR (about 160k locations) takes about 7 min.

5.6 Timeline

Phase 1: Download the remaining areas and OpenClimbing tiles. Collect GPS positions for the held-out HCR crag by hand. Check the pseudo-labels against the precise routes (distance-error CDF).

Phase 2: Train the 3D model under all modes, label settings and seeds. Train the 2D model and run the geometric baseline.

Phase 3: Tune the confidence threshold (MAPE and density), compare 2D and 3D, score both test sets, run the HCR probability map, and write up.

6 Collaboration & resources

6.1 GitHub

`https://github.com/euskon/705-643-final-project`

6.2 Hugging Face

`https://huggingface.co/euskon/en.705.643`

6.3 Collaboration Strategy

We will create our own branches and work in parallel on these branches to allow us to not clobber each others work. We will merge the branches back together as needed to a shared main trunk. We will both train different models, Zach will train the 3-D PointNet++ models on his machine and Yixian will train the 2-D ResNet-18 models on her machine. We will both upload to HuggingFace repository mentioned above at each iteration and/or checkpoint during training.

References

[1] U.S. Geological Survey. 3D Elevation Program (3DEP). `https://www.usgs.gov/3d-elevation-program`. Accessed Oct. 2026.

[2] Mountain Project. `https://www.mountainproject.com`. Accessed Oct. 2026.

[3] C. R. Qi, L. Yi, H. Su, L. J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. NeurIPS, 2017.

[4] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, J. Xiao. 3D ShapeNets: A deep representation for volumetric shapes. CVPR, 2015.

[5] N. Li, O. Kähler, N. Pfeifer. A comparison of deep learning methods for airborne lidar point clouds classification. IEEE JSTARS, 14:6467–6486, 2021.

[6] C. M. Albrecht et al. Learning and recognizing archeological features from LiDAR data. IEEE Big Data, 2019.

[7] M. U. Rafique, J. Zhu, N. Jacobs. Automatic segmentation of sinkholes using a convolutional neural network. Earth and Space Science, 2022.

[8] X. Yan. Pointnet/Pointnet++ Pytorch. `https://github.com/yanx27/Pointnet_Pointnet2_pytorch`, 2019.

[9] K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. CVPR, 2016.

[10] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, L. Fei-Fei. ImageNet: A large-scale hierarchical image database. CVPR, 2009.

[11] Open Climbing `https://openclimbing.org`. Accessed Oct. 2026.